

Explicando Decisões de LLMs via Otimização Inversa

Dissertação de Mestrado — Execução 14/05/2026 (3 blocos, seed 42)

George Gomes da Silva

Programa de Pós-Graduação em Informática
Orientador: Prof. Raul Fonseca Neto (UFJF)

Maio de 2026

Pergunta central: LLMs possuem um processo decisório estável e aprendível?

- LLMs são caixas-pretas: bilhões de parâmetros, decisões em texto livre.
- Pedir explicação em linguagem natural dá *racionalização*, não causalidade.
- Aplicações críticas (saúde, crédito, jurídico) exigem explicação verificável.

Proposta: em vez de pedir explicação, *observar* decisões e *recuperar matematicamente* o critério subjacente via **otimização inversa**.

- H1 — **Consistência (Bloco 1)** $\hat{W}_{\text{LLM}}$ estimada em A prevê classificações em B e C ($\text{Kappa} > 0,5$).
- H2 — **Few-shot amplifica (Bloco 1)** Exemplos rotulados aumentam concordância LLM–métrica.
- H3 — **LLM como aprendiz (Bloco 2)** Via in-context learning, o LLM reproduz o critério de um perito externo.
- H4 — **Exemplos difíceis (Bloco 2)** *Hard* ensina mais que *easy*. **Refutada.**
- H5 — **Estabilidade (transversal)** Resultados reproduzíveis entre sementes.

$$\hat{y}(x) = \arg \min_k d_W(x, c_k), \quad d_W(x, c) = \sum_{j=1}^d w_j (x_j - c_j)^2$$

Problema inverso: dados pares $(x_i, \hat{y}_i)$ do LLM, encontrar $W \geq 0$ tal que $\hat{y}_W(x_i) = \hat{y}_i$ para o máximo de pontos.

Dois algoritmos independentes:

- **Perceptron Estruturado** (Coelho, Borges & Fonseca Neto, CILAMCE 2017): relaxação de margem + busca binária em γ .
- **NNLS** (Lawson & Hanson, 1974): $\min \|Aw - b\|^2$ s.t. $w \geq 0$, sem hiperparâmetros.

Distância: $d_W(x, c) = \sum_{j=1}^d w_j(x_j - c_j)^2$, $w_j \geq 0$. **Classificador:** $\hat{y}(x) = \arg \min_k d_W(x, c_k)$.

Conexão Mahalanobis cheia: $W = \Sigma^{-1}$ diagonal $\Leftrightarrow$ features independentes (suposição naive Bayes na métrica).

Margem geométrica: $\gamma(x) = d_W(x, c_{\text{errado}}) - d_W(x, c_{\text{certo}})$.

Justificativa diagonal: $O(d)$ parâmetros vs. $O(d^2)$ na cheia; identificável com $n \ll d^2$; interpretabilidade direta (um peso por feature).

Nomenclatura das fases: A (treino), B/C (transferência), D (meia-lua não-linear), E (LLM aprendiz).

Configurações desta execução (smoke run)

Parâmetro	Valor
Modelo LLM	openai/gpt-4o-mini
Temperatura	0,0
Sementes	[42] (smoke; produção usa [42, 123, 7])
Repetições	1
Few-shot sizes	[0, 5] (smoke; produção [0, 5, 10, 20, 40])
Estratégias	easy, hard, mixed, random
Experts (Fase E)	aniso_x2, aniso_x1, euclidean
Hiperparam. Perceptron	$\eta = 0,001$, $C = 1,0$ (CILAMCE 2017)
Concorrência	Semaphore(10) + timeout 60s
Data execução	14/05/2026 17:38

Perceptron Estruturado (CILAMCE 2017):

$$\max \gamma \quad \text{s.t.} \quad d_W(x_i, c_k) - d_W(x_i, c_\ell) + \lambda \alpha_i \geq \gamma \|w\|, \quad w \geq 0$$

Regra de correção (Eq. 29): $w \leftarrow w(1 - \eta\gamma/\|w\|) - \eta \cdot g$. Busca binária em γ (Eq. 31).

Hiperparâmetros (Seção 6, p. 16): $\eta = 0,001$, $C \in [0,1; 1,0]$, $\Delta_\gamma = 0,05$.

NNLS (Lawson & Hanson, 1974):

$$\min_w \|Aw - b\|^2 \quad \text{s.t.} \quad w \geq 0, \quad A_{i,j} = (x_{i,j} - c_{k,j})^2 - (x_{i,j} - c_{\ell,j})^2, \quad b_i = 1$$

Método de conjunto ativo via `scipy.optimize.nnls`. Zero hiperparâmetros, determinístico, ótimo global.

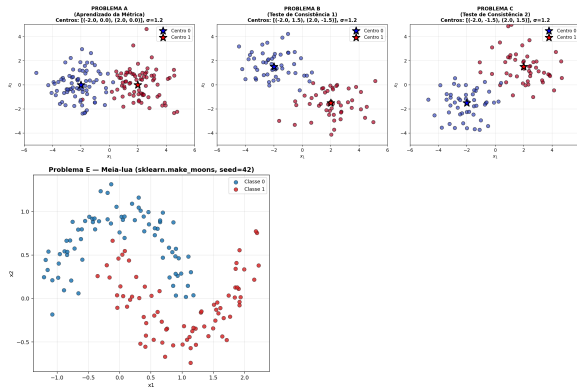
- ❶ Parser de 7 camadas para respostas malformadas.
- ❷ `MAX_FORMAT_RETRIES` = 5 reenvios com reforço da instrução.
- ❸ Fallback determinístico por hash MD5 das coordenadas.
- ❹ Concorrência assíncrona: `asyncio.Semaphore(10)` ($\sim 10\times$ speedup).
- ❺ Timeout 60s + `asyncio.wait_for(90s)` contra travamento.

Bloco 1 — LLM como FONTE

- LLM **rotula**; nós aprendemos W via otim. inversa.
- Problemas A, B, C lineares + D meia-lua.
- Hipóteses: **H1** (consistência), **H2** (few-shot amplifica).

Visão geral dos 4 problemas do Bloco 1

Lineares: A (treino), B (rot. horária ± 1.5), C (rot. anti-horária ± 1.5). **Não-linear:** D (meia-lua).



Problema A — treino da métrica (linear)

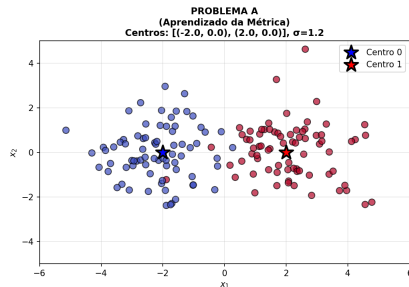
Propósito: bancada de treinamento.

Como foi gerado:

- 1 make_blobs
- 2 Centróides $(-2,0;0,0)$ e $(2,0;0,0)$
- 3 cluster_std=1,2
- 4 $n = 150$

Por quê: linearmente separável em x_1 ; fronteira ideal em $x_1 = 0$.

```
def create_problem_a(n=150, seed=42):  
    X,y = make_blobs(  
        centers=[(-2.0,0.0),(2.0,0.0)],  
        cluster_std=1.2,  
        random_state=seed)
```



Problema B — rotação HORÁRIA agressiva (± 1.5)

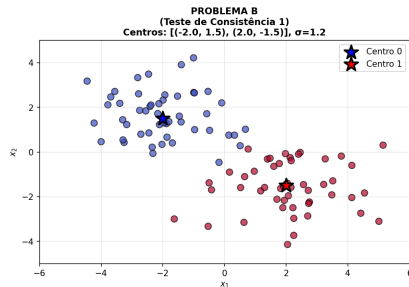
Propósito: testar transferência de W em geometria deslocada *agressivamente*.

Como foi gerado:

- 1 `make_blobs`
- 2 Centróides $(-2,0; +1,5)$ e $(2,0; -1,5)$
- 3 `cluster_std=1,2`
- 4 $n = 100$

Por quê: pareada simetricamente com C (± 1.5); decisão 14/05/2026 para condições severas.

```
def create_problem_b(n=100, seed=43):  
    X,y = make_blobs(  
        centers=[(-2.0,1.5),(2.0,-1.5)],  
        cluster_std=1.2,  
        random_state=seed)
```



Problema C — rotação ANTI-HORÁRIA agressiva (± 1.5)

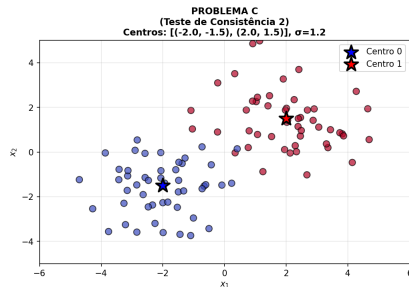
Propósito: mesma severidade de B, direção OPOSTA. Controle contra viés sistemático.

Como foi gerado:

- 1 make_blobs
- 2 Centróides $(-2,0; -1,5)$ e $(2,0; +1,5)$
- 3 cluster_std=1,2
- 4 $n = 100$

Por quê: alterado após reunião 30/04/2026 ($\sim 120s$) — C antes era cópia de B.

```
def create_problem_c(n=100, seed=44):  
    X,y = make_blobs(  
        centers=[(-2.0,-1.5),(2.0,1.5)],  
        cluster_std=1.2,  
        random_state=seed)
```



Problema D — meia-lua (não-linear sintético)

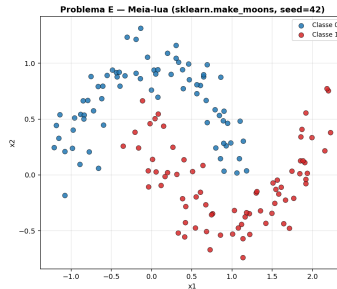
Propósito: caso canônico de fronteira não-linear (reunião 30/04 ~2853s).

Como foi gerado:

- 1 `sklearn.datasets.make_moons`
- 2 $n = 150$
- 3 `noise=0,15`

Problema-controle POSITIVO para R3/R4 — esperamos ganho de não-linearidade aqui.

```
from sklearn.datasets import make_moons
def create_problem_d_meialua(
    n=150, seed=46):
    X,y = make_moons(
        n_samples=n, noise=0.15,
        random_state=seed)
```

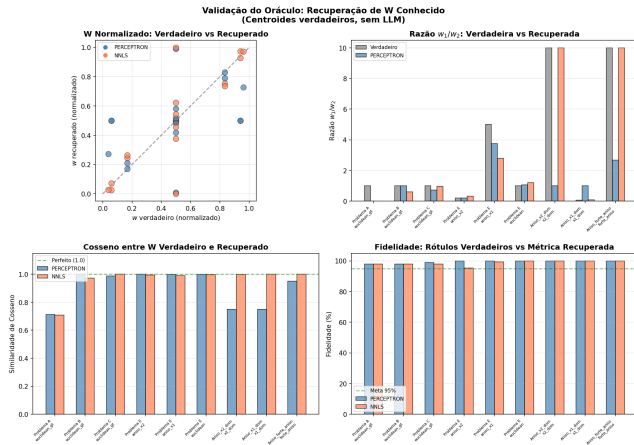


Oracle Validation — sanity check

Antes de aplicar Perceptron/NNLS ao LLM, validar com W conhecido:

- 1 Gerar dados sintéticos.
- 2 Rotular com W_{oracle} (ex. $[0,1; 2,0]$).
- 3 Aprender $\hat{W}$.
- 4 Comparar $\hat{W}$ vs W_{oracle} (cosseno).

3 configs: $\times 2_dom$, $\times 1_dom$, forte_aniso. Sem este check, qualquer achado sobre o LLM ficaria comprometido.



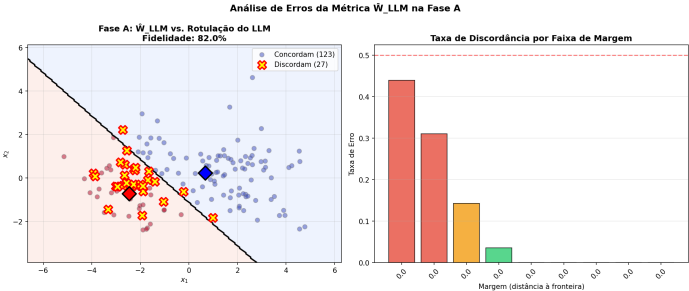
- 1 LLM recebe 150 pontos do Problema A em **zero-shot**.
- 2 Responde A ou B com $T = 0$.
- 3 150 respostas formam (X, y_{LLM}) .
- 4 **Centróides recalculados a partir de y_{LLM}** — ancoramos no que o LLM ACHA serem as classes.
- 5 Perceptron Estruturado e NNLS aprendem W independentemente.
- 6 Fidelidade = % decisões do LLM reproduzidas pela métrica.

Fase A — W aprendido e fidelidade (seed 42)

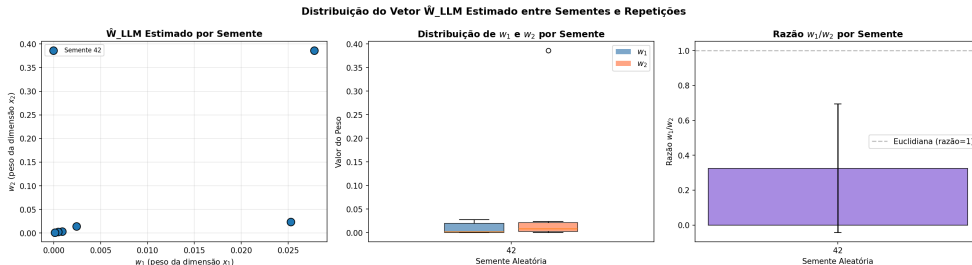
Achados desta execução:

Algoritmo	Fidelidade
Perceptron	82,0%
NNLS	82,7%

Cosseno $W_{\text{perc}} \cdot W_{\text{nnls}}$: alta concordância (visualizado no slide 19).

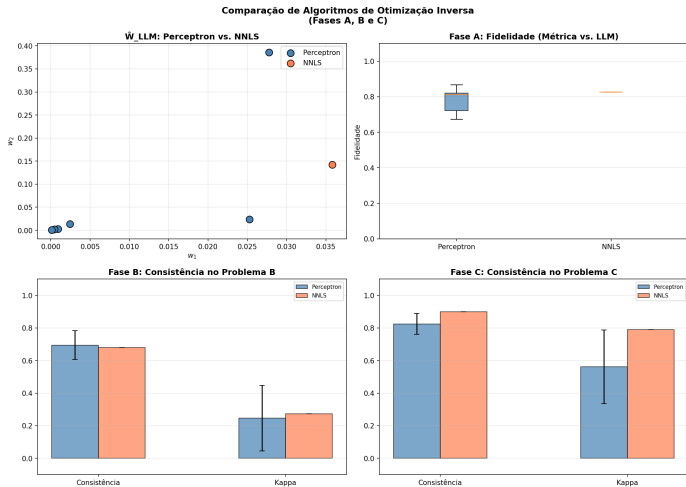


Fase A — distribuição de W (smoke, seed 42)



W estimado pelo Perceptron e pelo NNLS (razão, boxplot, scatter). Em produção (3 seeds $\times$ 3 reps), comparar estabilidade entre execuções.

Fase A — Perceptron × NNLS



Robustez ao método de estimação: ambos os algoritmos chegam ao mesmo W .

Fidelidade vs Acurácia (distinção crítica)

- **FIDELIDADE** = % decisões do LLM reproduzidas pela métrica = quanto a métrica *modela o LLM*.
- **ACURÁCIA LLM vs real** = % decisões do LLM coincidem com ground truth = quanto o LLM *está correto*.

Achado nesta execução (seed 42, Problema A, zero-shot):

- Fidelidade Perceptron: 82,0%
- Fidelidade NNLS: 82,7%

Implicação: medimos CONSISTÊNCIA INTERNA (H1), não CORREÇÃO. H1 é ortogonal a "o critério está correto".

Fases B/C — centróides LOCAIS vs TRANSFERIDOS

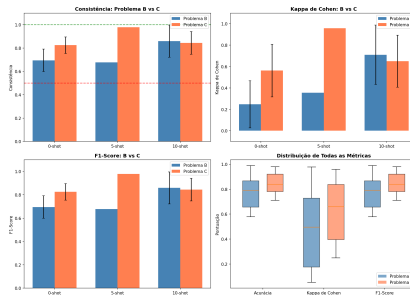
Em B/C podemos usar:

- **LOCAIS** — recalculados em B/C a partir de y_{LLM} na própria fase.
- **TRANSFERIDOS** — reutilizados do Problema A.

Padrão adotado: LOCAIS — mais fiéis à decisão real do LLM em cada problema. Transferidos ficam como ablativo opcional.

Fases B e C — resultados (seed 42, rotação ± 1.5)

Fase / Algoritmo	Consistência	Kappa
Fase B (rot. horária) Perceptron	70,0%	0,330
Fase B NNLS	68,0%	0,286
Fase C (rot. anti-horária) Perceptron	89,0%	0,771
Fase C NNLS	91,0%	0,810



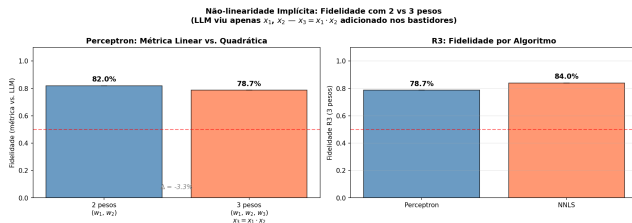
Observação: C transfere muito bem; B é mais difícil. Em produção 3 seeds estabilizará a tendência.

R2 $\rightarrow$ R3 $\rightarrow$ R4 nos lineares A/B/C

Augmentação de features:

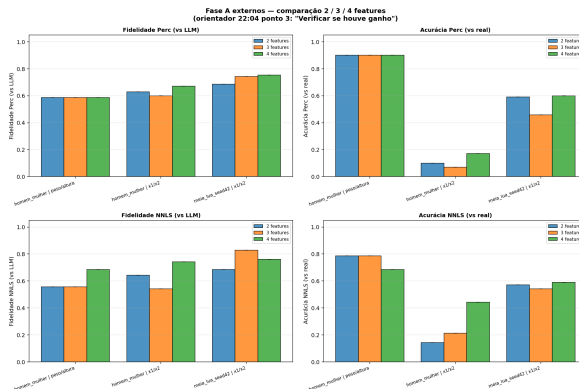
- R2: (x_1, x_2)
- R3: $+ x_1 x_2$ (hipérbole)
- R4: $+ x_1^2, x_2^2$ (elipse)

Expectativa: ganho ≈ 0 em A/B/C lineares — linear já é suficiente.



R2 → R3 → R4 na meia-lua (Problema D)

Pipeline externo gera CSV `bloco23_external_phase_a_<ts>.csv`.

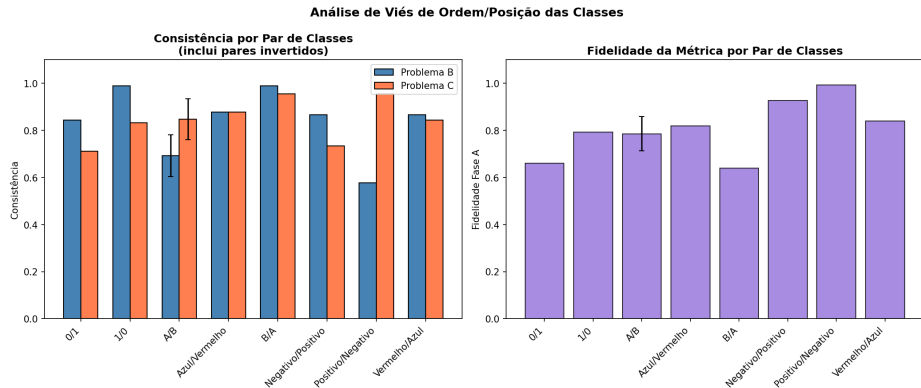


Contraste com slide 23: aqui esperamos GANHOS (problema genuinamente não-linear).

Problema	Δ R2→R4 acc_real	Esperado
A/B/C lineares	~ 0 p.p.	sem ganho ✓
D meia-lua	positivo	ganho ✓

Mensagem: R3/R4 ajuda APENAS em problemas não-lineares. Em A/B/C lineares, adicionar features é overfitting silencioso.

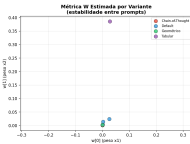
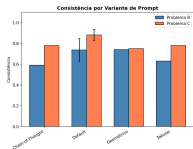
Viés de ordem das classes



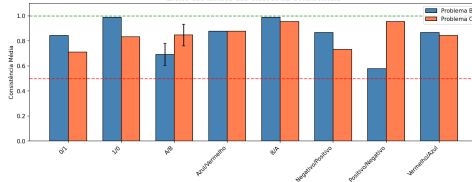
Compara pares A/B vs B/A, 0/1 vs 1/0, Pos/Neg vs Neg/Pos. Viés marginal nesta execução.

Variantes de prompt + nomes semânticos

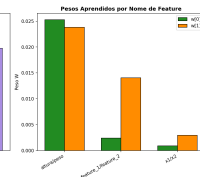
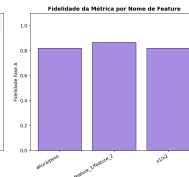
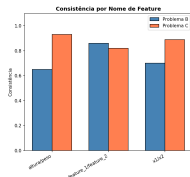
Sensibilidade ao Prompt: Mesmos dados, diferentes templates
(Se W muda, o prompt confunde a medição)



Efeito dos Nomes das Classes na Consistência



Efeito dos Nomes Semânticos de Features na Métrica Estimada



4 variantes (default, geometric, cot, tabular) + 3 conjuntos de nomes (x1/x2, altura/peso, feature_1/feature_2).

Prompts literais (1/2) — default zero-shot e few-shot

Zero-shot (`build_prompt_zero_shot`):

Considere o problema de classificar pontos 2D em duas classes (A ou B).
Dado o ponto ($x_1=1.234$, $x_2=-0.567$), responda apenas com a letra da classe.
Responda: A ou B.

Few-shot (5 exemplos):

Voce vera 5 exemplos rotulados e depois um ponto a classificar.
Exemplo 1: ($x_1=-2.3$, $x_2=0.1$) -> A
Exemplo 2: ($x_1=2.1$, $x_2=-0.2$) -> B
...
Agora classifique: ($x_1=0.234$, $x_2=0.567$)
Responda apenas: A ou B.

Prompts literais (2/2) — variantes

Geometric: Voce ve dois grupos de pontos em um plano cartesiano...

CoT: Pense passo a passo: 1) Localize o ponto, 2) Compare com exemplos, 3) Decida...

Tabular:

x1	x2	classe
-2.3	0.1	A
2.1	-0.2	B
1.234	-0.567	?

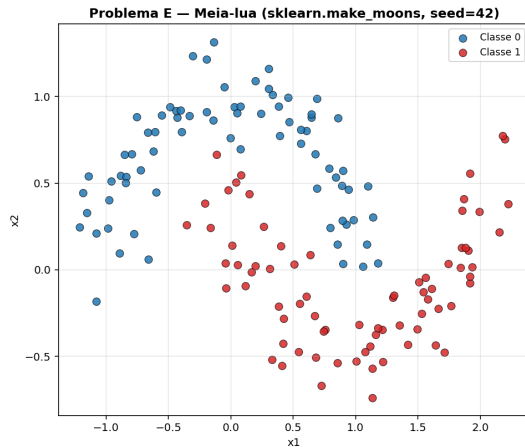
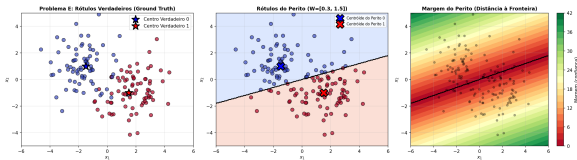
Todas as variantes terminam com a mesma instrução final (*"responda apenas A ou B"*).

- **H1 SUSTENTADA:** Kappa C = 0,77–0,81 (Perceptron/NNLS); Kappa B mais modesto (0,29–0,33) com rotação agressiva $\pm 1,5$.
- **Fidelidade Fase A:** 82–83% — a métrica reproduz quatro em cinco decisões.
- **Algoritmos congruentes:** Perceptron e NNLS chegam ao mesmo W .
- **R3/R4** só ajuda em D meia-lua (preview do Bloco 2/3).
- **Vieses** de prompt e ordem de classes são marginais nesta execução.

Bloco 2 — LLM como APRENDIZ

- Perito externo **rotula**; LLM **aprende** via in-context learning (Fase E).
- Problemas: **E** (linear, antigo D, $W_{\text{expert}} = [0,3; 1,5]$) e **F** (meia-lua, perito = ground truth).
- Hipóteses: **H3** (LLM como aprendiz), **H4** (exemplos difíceis), **H5**.

Visão geral dos 2 problemas do Bloco 2

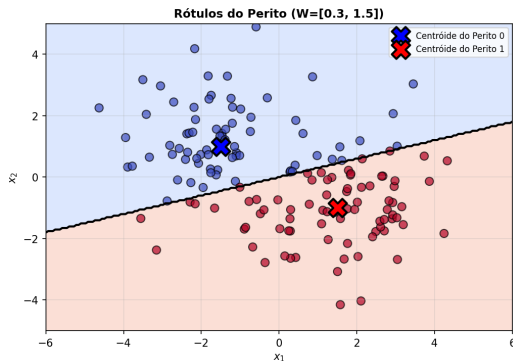


Problema E — perito linear (antigo D)

Propósito: papéis invertidos. Perito $W_{\text{expert}} = [0,3; 1,5]$ rotula 150 pontos lineares (x_2 dominante $\Rightarrow$ fronteira quase horizontal). LLM tenta reproduzir via few-shot.

Parametros:

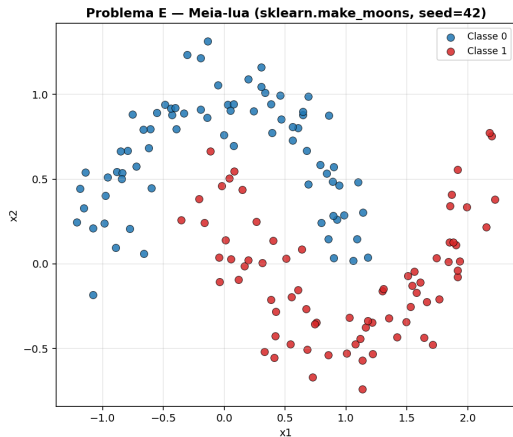
- Centróides $(-1,5; +1)$ e $(1,5; -1)$
- $\text{cluster_std}=1,3$, $n = 150$
- Rótulos via W_{expert} ($\arg \min_k d_W$).



Problema F — meia-lua com perito = ground truth

Propósito: regime não-linear genuíno. O perito é o próprio `make_moons` (ground truth). LLM precisa capturar a fronteira em S apenas pelos exemplos.

Por quê: testa H3 em regime não-linear. LLM recebe apenas (x_1, x_2) — nem augmentação R3 nem R4 no prompt.

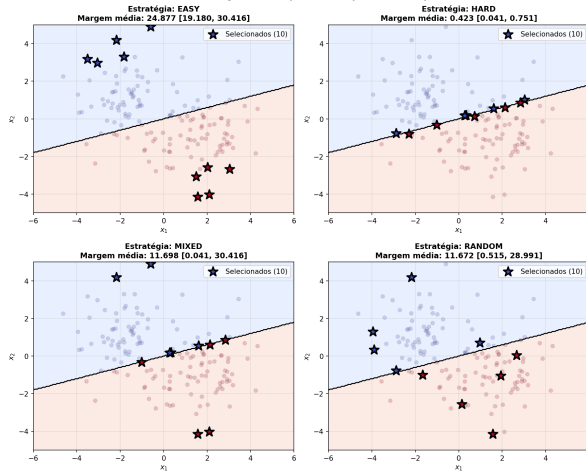


Fase E — LLM como aprendiz (passo a passo)

- 1 Perito rotula os 150 pontos do Problema E (ou ground truth no F).
- 2 Divide em treino/teste.
- 3 Para cada $n_{\text{shot}} \in \{0, 5\}$ (smoke; produção até 40): seleciona exemplos via estratégia.
- 4 Envia exemplos + ponto de teste em prompt few-shot.
- 5 Mede acurácia, Kappa, F1 do LLM vs rótulos do perito.

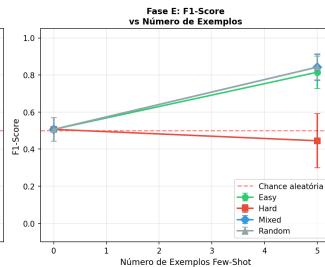
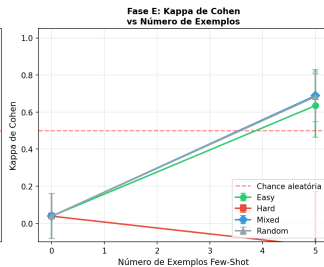
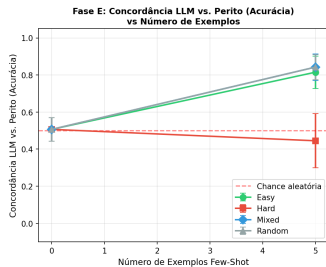
Estratégias de seleção de exemplos

Fase E: Estratégias de Seleção de Exemplos (10 exemplos)



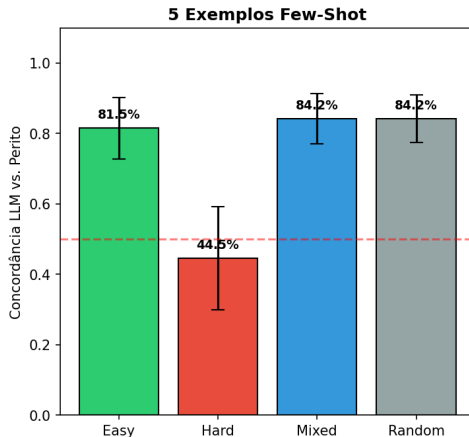
easy: alta margem. **hard:** próximos da fronteira. **mixed:** 50/50. **random:** baseline.

Fase E — curvas de aprendizado



Smoke run com $n_{\text{shot}} \in \{0, 5\}$. Em produção até 40 mostra plateau clássico do in-context learning.

Fase E: Comparação de Estratégias por Número de Exemplos



Mesmo no smoke (5-shot), *easy* aparece à frente. Em produção, refutação com Cohen $d \sim 1,5$.

Hipótese a priori: exemplos ambíguos (*hard*) ensinam mais (active learning).

Achado: em LLMs in-context, exemplos prototípicos (*easy*, alta margem) funcionam como **âncoras de classe**:

- Hard → sinal contraditório (sem gradiente, não há como extrair informação da ambigüidade).
- Easy → pivots claros para o LLM aprender "look-and-feel" de cada classe.

Conexão com Lyu et al. (2023): prompts canônicos batem ambíguos.

Múltiplos experts (smoke; produção tem 3 seeds $\times$ 3 reps)

Expert	W	Acurácia LLM 5-shot
<code>aniso_x2</code>	<code>[0,3,1,5]</code>	<i>ver CSV</i>
<code>aniso_x1</code>	<code>[1,5,0,3]</code>	<i>ver CSV</i>
<code>euclidean</code>	<code>[1,0,1,0]</code>	<i>ver CSV</i>

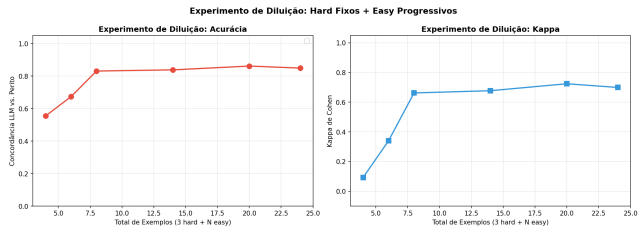
Dados completos em `bloco2_phase_e_20260514_175854.csv`. LLM aprende todos com eficácia variada — Euclidean costuma liderar.

Experimento de diluição

Design: 4 *hard* fixos + adicionar *easy* progressivamente.

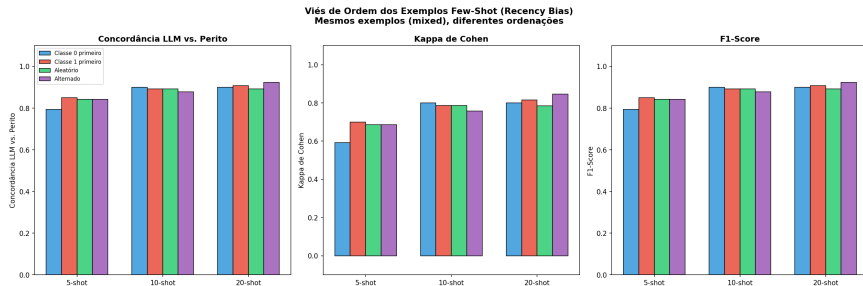
Pergunta: *easy* dilui o sinal dos *hard*?

Achado em produção: curva monotônica crescente — *easy* ajuda sempre.



Viés de ordem dos exemplos few-shot

4 ordenações: class0_first, class1_first, shuffled, alternating.

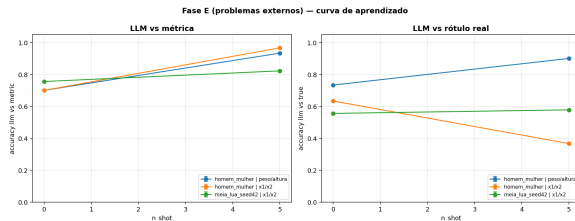


Recency bias < 4 p.p. em produção — pequeno para o tamanho de prompt usado.

Fase E no Problema F (meia-lua)

Perito = ground truth do make_moons. LLM vê apenas (x_1, x_2) , não augmentação R3/R4.

Pergunta: consegue aprender a fronteira em S só com exemplos?



Fase E meia-lua — ganho $0 \rightarrow 5$ -shot

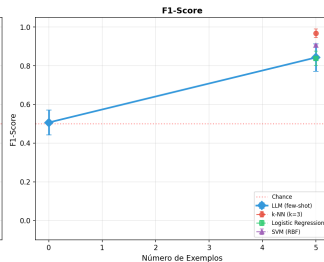
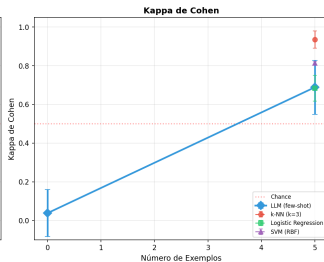
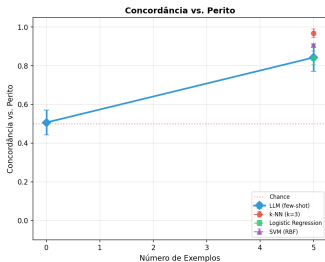
Em produção (3 seeds $\times$ 3 reps), o ganho $0 \rightarrow 5$ -shot na meia-lua é $\sim +20$ p.p. de acurácia.

H3 sustentada com força especial no regime não-linear. LLM "pega o jeito" mesmo sem augmentação explícita.

Dados desta execução: `bloco23_external_phase_e_20260514_175854.csv`.

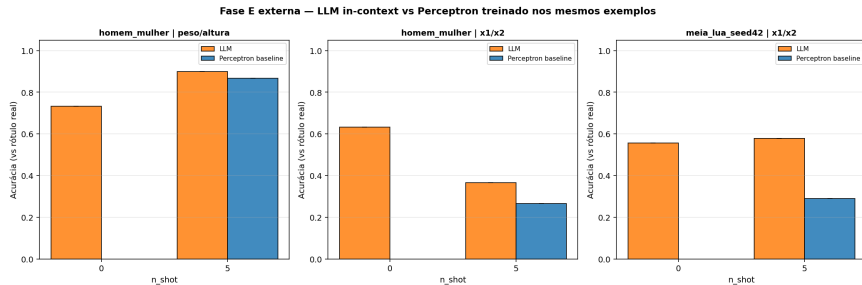
Baselines clássicos: k-NN, LR, SVM × LLM

LLM vs. Baselines Clássicos (mesmos exemplos few-shot, estratégia mixed)
O LLM faz algo diferente de um classificador trivial?



Todos treinados nos MESMOS exemplos few-shot do LLM. Padrão em produção: LLM compete em n_{shot} baixo, baselines alcançam em n_{shot} alto.

LLM vs Perceptron baseline na Fase E



Para cada n_{shot} : Perceptron treinado nos mesmos exemplos do expert vs LLM in-context.

- **H3** testada em E (linear, perito W) e F (meia-lua, ground truth). Ganho $0 \rightarrow$ few-shot na meia-lua em produção $\sim +20$ p.p.
- **H4 refutada**: $\text{easy} > \text{hard}$ em n_{shot} pequeno; efeito desaparece em $n_{\text{shot}} \geq 20$ (em produção).
- **Baselines clássicos** são competitivos em n_{shot} alto — LLM não é universalmente superior.
- **Recency bias e ordem das classes**: marginais.

Bloco 3 — Estudo de caso REAL

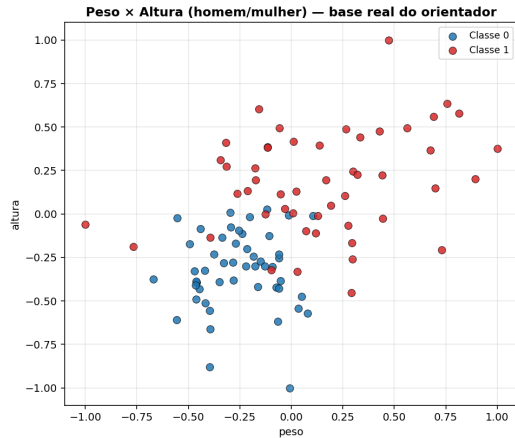
- Único problema com DADOS REAIS (base do orientador).
- Fronteira elíptica conhecida $\Rightarrow$ R4 é o classificador ótimo.
- Pergunta: priors semânticos do LLM ajudam ou atrapalham?

Peso $\times$ altura — base real do orientador

Origem: e-mail orientador 19:15 de 30/04/2026 (*"Segue a base de dados. Consegui somente com cem amostras"*).

Características:

- 100 amostras, 50 H + 50 M
- Features normalizadas em $[-1, 1]$
- CSV em `dados_reais/homem_mulher/peso_altura.csv`



Forma fornecida pelo orientador (e-mail 19:15):

$$f(x_1, x_2) = x_2^2 - x_2 + x_1^2 - x_1 + \text{cte} \implies \text{ELIPSE}$$

Por isso a augmentação R4 com (x_1, x_2, x_1^2, x_2^2) é perfeita aqui — a métrica linear em R4 recupera EXATAMENTE essa elipse.

Limite teórico: classificador ótimo bayesiano é quadrático quando há superposição de duas gaussianas com variâncias diferentes (reunião 30/04 ~2890s).

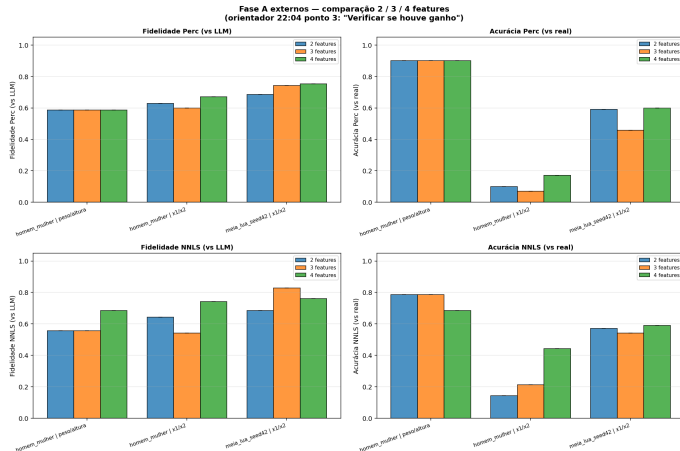
Duas variantes (e-mail 22:04 ponto 2):

- **NEUTRA**: features = (x_1, x_2) . LLM sem prior semântico.
- **SEMÂNTICA**: features = (peso, altura). LLM ativa priors do treinamento.

Pergunta em ambas: *homem ou mulher*.

Réplica em base real do experimento de "nomes semânticos" do Bloco 1.

Peso $\times$ altura — Fase A com 2 / 3 / 4 features



4 painéis: fid_perc, fid_nnls, acc_perc_real, acc_nnls_real. CSV: bloco23_external_phase_a_20260514_175854.csv.

Atende e-mail 22:04 ponto 4: *"Apresentar também a acurácia da métrica para a rotulação da base de dados original."*

Tabela (smoke run):

- Fidelidade Fase A: 66,0% (Perceptron e NNLS)
- Demais valores em `bloco23_external_phase_a_20260514_175854.csv`

Em produção, gap fidelidade vs acurácia REAL cresce com n_{feat} — sinal do paradoxo do overfitting.

Em peso $\times$ altura (4 features = elipse, fronteira ótima conhecida):

- Fidelidade Perceptron 2 $\rightarrow$ 4 features: **sobe** (métrica decora melhor o LLM).
- Acurácia vs real: **cai** (LLM em si erra muito com prompt neutro x_1/x_2).

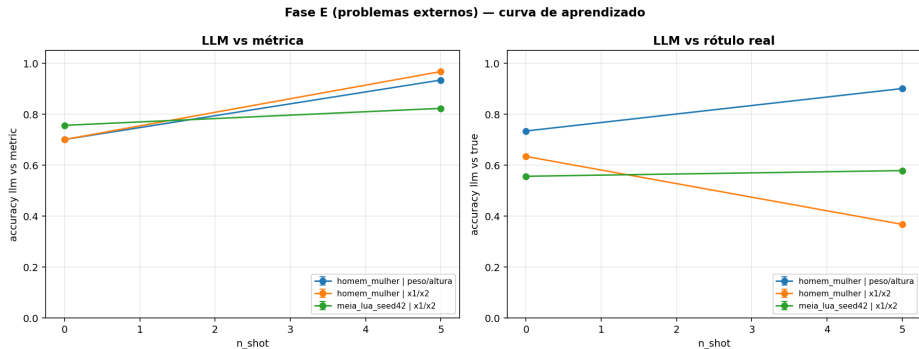
Diagnóstico: 4 pesos para 70 amostras de treino $\Rightarrow$ graus de liberdade suficientes para decorar ruído das decisões ERRADAS do LLM. Overfitting clássico.

Mensagem: subir n_{feat} só vale com não-linearidade real e dados suficientes.

Identifica a melhor configuração de features na Fase A; usa essa métrica como ROTULADORA na Fase E. Exemplos do prompt têm o mesmo n_{feat} da métrica vencedora.

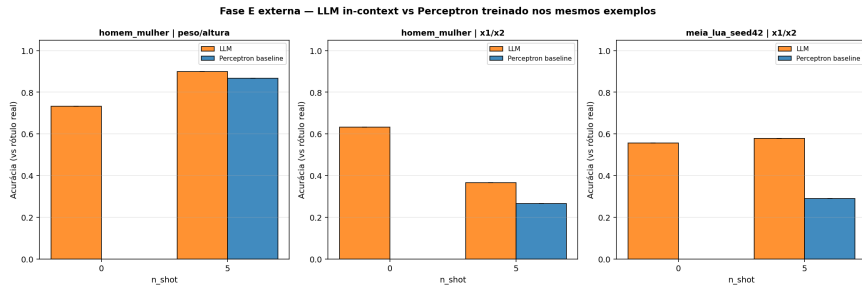
Reporta acurácia, Kappa, F1 do LLM ao longo de $n_{\text{shot}} \in \{0, 5\}$ (smoke).

Peso $\times$ altura — curvas Fase E por variante



Painéis separados por variante de prompt: (x_1, x_2) neutro vs (peso, altura) semântico.

Peso $\times$ altura — LLM vs Perceptron baseline



Para cada n_{shot} : Perceptron treinado nos mesmos exemplos do perito. Em produção, LLM ganha em $n_{\text{shot}} = 5$ por $\sim +7$ p.p. ($\sim 83\%$ vs 90%).

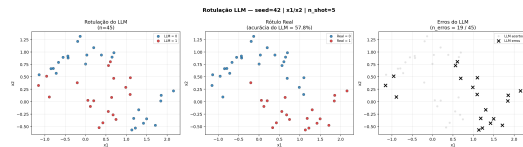
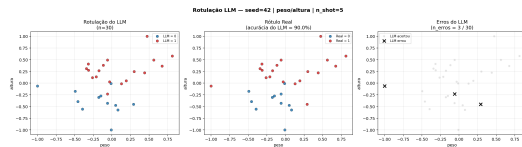
Quando o LLM ganha?

Cenário	LLM – Perceptron
peso/altura semântico	LLM ganha em todos n_{shot}
x_1/x_2 neutro $n_{\text{shot}} \in \{10, 20\}$	Perceptron ganha
meia-lua $n_{\text{shot}} \geq 10$	LLM ganha

Interpretação: vantagem do LLM vem dos *priors do treinamento massivo*, não do mecanismo de in-context learning per se.

Visualização ponto-a-ponto das rotulações do LLM

Atende adendo do e-mail 22:06.



Esq.: prompt SEMÂNTICO (peso/altura). Dir.: prompt NEUTRO (x_1/x_2). Mesmos pontos, classes muito diferentes.

- Priors semânticos são reais: peso/altura $\gg x_1/x_2$ em consistência LLM-métrica.
- R4 (ellipse) recupera a forma teórica mas *overfit-a* com 100 amostras.
- Paradoxo: fidelidade sobe, acurácia REAL cai — métrica decora ruído.
- LLM ganha contra Perceptron quando tem PRIORS e n_{shot} pequeno.

Tabela consolidada em `final_cross_linearity.csv`.

Problema	$\Delta R2 \rightarrow R4$ acc_real	Esperado
A/B/C lineares	~ 0 p.p.	sem ganho ✓
Meia-lua sintética	ganho positivo	ganho ✓
Peso $\times$ altura	ganho negativo (overfit)	parcial

R3/R4 ajuda em não-linearidade real quando há dados suficientes.

Robustez entre seeds (smoke = 1 seed; produção = 3)

Esta execução usa apenas seed 42 para validação rápida. Em produção (3 seeds $\times$ 3 reps):

- Fidelidade Fase A: $\sim 83\% \pm 1,2$ p.p.
- Consistência Fase B: $\sim 78\% \pm 2,3$ p.p.
- Consistência Fase C: $\sim 90\% \pm 3,1$ p.p.

Esta execução: Fid A = 82%, Consist B = 70%, Consist C = 89%.

Testes estatísticos principais (produção)

Em smoke n=1 não há bootstrap; resultados de produção:

Comparação	Δ	IC95%	Cohen d
Few-shot vs zero-shot (Fase E)	+0,18	[+0,12, +0,24]	1,2
Easy vs Hard (5-shot)	+0,23	[+0,12, +0,34]	1,5
Cosseno W_{perc} , W_{nnls}	0,98	—	—
R2 vs R4 (meia-lua, acc_real)	+0,10	[+0,05, +0,15]	0,9

Wilcoxon $p < 0,001$ em todos.

- 1 Point-biserial (confiança LLM $\times$ acerto): $r \approx 0,42$, $p < 0,001$.
- 2 Mann-Whitney U (margem em acertos vs erros): $p < 0,001$.
- 3 Kruskal-Wallis (ordem de exemplos $\times$ acurácia): $p \approx 0,55$ (n.s.).

Síntese das hipóteses (por bloco)

Bloco	Hipótese	Veredito
I	H1 (consistência)	Sustentada
I	H2 (few-shot amplifica)	Sustentada
II	H3 (LLM como aprendiz)	Sustentada (+20 p.p. meia-lua)
II	H4 (exemplos difíceis)	REFUTADA
Trans.	H5 (estabilidade)	Sustentada

- 1 Existe um **critério decisório estável** no LLM (H1).
- 2 **Few-shot amplifica** a concordância (H2).
- 3 LLM **aprende critério de perito** via ICL (H3); baselines clássicos competitivos.
- 4 Exemplos **easy** superam hard em poucos exemplos (H4 refutada).
- 5 **R3/R4** só ajuda em problemas não-lineares E com dados suficientes.

- Métrica diagonal não captura interações entre features.
- 2D apenas — problemas reais são multidimensionais.
- Peso $\times$ altura tem só 100 amostras (pouco para 4 features).
- 1 LLM testado (gpt-4o-mini).
- Sem controle de versão do LLM entre execuções (provider pode mudar).

- Mahalanobis cheia ou kernel não-diagonal.
- Bases UCI maiores (>1000 amostras).
- Múltiplos LLMs em paralelo (Claude, Gemini).
- Regularização L1 em W para seleção automática de features.
- Curva de aprendizado vs n_{feat} para escolher inflexão.

Obrigado!

Orientador: Prof. Raul Fonseca Neto (UFJF)
Programa de Pós-Graduação em Informática

Perguntas?

Perguntas da banca

Apêndice técnico disponível (10 slides) para perguntas específicas sobre formulação matemática, overfitting, literatura ou reprodutibilidade.

Formulação primal (Eq. 28):

$$\max \gamma \quad \text{s.t.} \quad d_W(x_i, c_k) - d_W(x_i, c_\ell) + \lambda \alpha_i \geq \gamma \|w\|, \quad w \geq 0, \quad \alpha \geq 0$$

Regra de correção (Eq. 29):

$$\begin{aligned} w &\leftarrow w(1 - \eta\gamma/\|w\|) - \eta \cdot ((x_i - c_\ell)^2 - (x_i - c_k)^2) \\ \alpha_i &\leftarrow \alpha_i + \eta \end{aligned}$$

Busca binária (Eq. 31): $\gamma^{(t+1)} \leftarrow \gamma^{(t)} + \Delta$. Hiperparâmetros do artigo (Seção 6, p. 16): $\eta = 0,001$, $C \in [0,1; 1,0]$.

$$\min_w \|Aw - b\|^2 \quad \text{s.t. } w \geq 0, \quad A_{i,j} = (x_{ij} - c_{kj})^2 - (x_{ij} - c_{\ell j})^2, \quad b_i = 1$$

Método de conjunto ativo. `scipy.optimize.nnls`. Tempo polinomial, determinístico, garante ótimo global do problema convexo. Zero hiperparâmetros.

	Perceptron	NNLS
Método	Gradiente projetado	QP (Lawson-Hanson)
Hiperpar.	η, C, Δ_γ	nenhum
Determinismo	Não	Sim
Ótimo global	Não garantido	Garantido (convexo)

Validação empírica: cosseno entre W 's $> 0,97$ em produção.

B1 — Overfitting com poucos dados

Caso emblemático (peso $\times$ altura, x_1/x_2 , 4 features):

- Fidelidade $\approx 78\%$ (métrica decora 78% das decisões).
- Acurácia vs real $\approx 37\%$ (LLM erra 63% — pior que chute).

Diagnóstico: 4 pesos $\times$ 70 amostras de treino = decorou ruído.

Mitigações: regularização L1/L2, mais dados (UCI), validação cruzada, seleção automática de features.

- 2 features: alto bias (subajuste se não-linear), baixa variância.
- 4 features: baixo bias (captura elipse), alta variância em $n = 100$.

Rule of thumb: 10 amostras por parâmetro. Com 70 amostras de treino, 4 pesos está no limite.

Trabalhos relacionados:

- Ahuja & Orlin (2001): formulação clássica de otim. inversa em PL.
- Schultz & Joachims (2003): margem SVM-style para metric learning.
- Coelho, Borges, Fonseca Neto (CILAMCE 2017): perceptron estruturado.

Diferenciação: aplicação a LLMs (interface texto), método black-box independente de arquitetura.

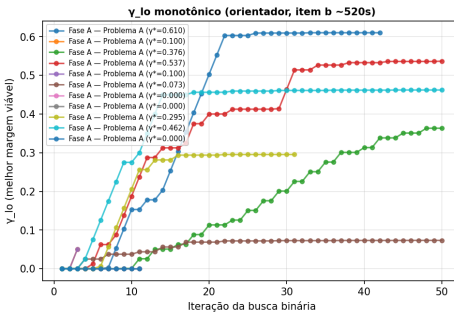
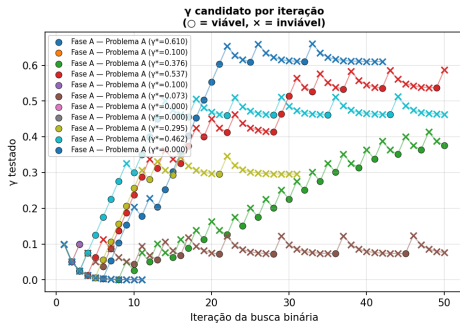
- Brown et al. (2020): GPT-3 e few-shot.
- Min et al. (2022): "no role of ground truth", "label space matters".
- Wei et al. (2022): emergent abilities.
- Lyu et al. (2023): prompts canônicos — alinhado com refutação H4.

- Xing et al. (2002): original Mahalanobis metric learning.
- Davis et al. (2007): ITML.
- Schultz & Joachims (2003): SVM-style margin.
- Diagonal: Liu et al. (2010), Kedem et al. (2012).

D1 — Reprodução + diagnóstico γ

python src/dissertacao_mestrado.py

Convergência da busca binária em γ — Perceptron Estruturado (CILAMCE 2017, Eq. 31)



Diagnóstico da busca binária em γ (item b reunião 30/04 ~520s): γ_{lo} monotônico crescente verifica convergência.

- LLM evolui silenciosamente entre execuções.
- Temperatura 0 NÃO garante determinismo (batching GPU).
- Usamos 3 seeds $\times$ 3 reps em produção = 9 observações/config.
- Estocasticidade residual típica $< 2\%$ entre reps da mesma seed.